WEEK 10 · PHASE 4

CI/CD on Databricks

Git Folders · DABs · Env Promotion · CLI

- Git Folders: branches, commits, PRs, repo sync
- Declarative Automation Bundles (DABs) structure
- databricks.yml — bundle configuration
- Environment targets: dev / staging / prod overrides
- Databricks CLI: key commands
- Practice Questions (10 Qs)

Table of Contents

1. CI/CD Overview on Databricks 3
2. Databricks Git Folders 4
3. Declarative Automation Bundles (DABs) 6
4. databricks.yml — Bundle Configuration 8
5. Environment Targets & Promotion 11
6. Databricks CLI 13
7. CI/CD Pipeline Patterns 15
8. Practice Questions 16
9. Answer Key 19

WEEK 10 **CI/CD Overview on Databricks** Git · DABs · CLI · Environments

1. CI/CD Overview on Databricks

CI/CD (Continuous Integration / Continuous Deployment) on Databricks enables teams to version-control notebooks and pipelines, test changes in isolated environments, and deploy to production automatically. Databricks provides two primary tools: Git Folders for source control integration and DABs for declarative deployment.

Tool	Purpose	Key Capability
Git Folders	Source control integration in workspace	Browse repos, create branches, commit, PR, sync
Declarative Automation Bundles (DABs)	Define & deploy Databricks resources as code	Jobs, pipelines, clusters defined in YAML; deploy via CLI
Databricks CLI	Command-line interface for all Databricks APIs	Deploy bundles, run jobs, manage workspace, automate
Databricks Asset Bundles	Another name for DABs (same thing)	Used interchangeably in docs

CI/CD Stage	What Happens	Tooling
Develop	Write code in feature branch in Git Folder	Git Folders, notebooks, Repos
Integrate (CI)	PR triggers automated tests, linting, unit tests	GitHub Actions / Azure DevOps + Databricks CLI
Deploy (CD) to Staging	DABs deploy to staging workspace/target	databricks bundle deploy --target staging
Test	Run integration tests on staging	databricks bundle run --target staging
Deploy (CD) to Prod	Promote bundle to production target	databricks bundle deploy --target prod

2. Databricks Git Folders

Git Folders (formerly Repos) integrate Git repositories directly into the Databricks workspace. Engineers work with notebooks as files in a Git repo, enabling version control, branching, and pull requests without leaving Databricks.

2.1 Supported Git Providers

Provider	Notes
GitHub (github.com)	Most common; GitHub Actions CI/CD integration
GitHub Enterprise Server	On-premises GitHub
GitLab	gitlab.com and self-hosted
Azure DevOps (Azure Repos)	Common in Azure-based deployments
Bitbucket	Atlassian Bitbucket Cloud and Server
AWS CodeCommit	AWS-native Git service

2.2 Core Git Operations in Databricks

Operation	Description	Where
Clone	Connect workspace to a remote Git repo	Repos UI → Add Repo → URL
Branch	Create/switch branches	Repo UI → branch selector
Commit	Save changes with a message	Repo UI → commit button
Push	Push local commits to remote	Repo UI → push
Pull	Fetch & merge remote changes	Repo UI → pull
Pull Request	Propose branch merge in Git provider	GitHub/GitLab/ADO UI
Sync	Fetch latest from remote	Repo UI → sync (pull + push)

2.3 Branch Strategy for Data Engineering

■ **Branch strategy**

```
# Recommended branch strategy:

# main (or master) ← Production code
#   ↑ PR merge after staging tests pass
# staging          ← Integration test environment
#   ↑ PR merge after unit tests pass
# feature/DE-123   ← Developer feature branch
#   Created from main, deleted after merge

# In Databricks Git Folder:
# Each environment uses a different branch/tag
# Dev workspace → feature branch
# Staging workspace → staging branch
# Prod workspace → main branch (tag)
```

■ **Note** Git Folders are READ/WRITE in the workspace. Changes made in a notebook are tracked as file changes in the Git repo — commit and push to persist them.

3. Declarative Automation Bundles (DABs)

DABs (Databricks Asset Bundles) let you define Databricks resources — jobs, pipelines, clusters, permissions — as YAML configuration files. Deploy using the Databricks CLI. DABs replace manual UI configuration for repeatable, environment-aware deployments.

3.1 What DABs Can Define

Resource Type	DABs Key	Example Use
Jobs (Workflows)	resources.jobs	Define ETL pipelines as code
DLT Pipelines	resources.pipelines	Define streaming/batch DLT pipelines
Clusters	resources.clusters	Shared cluster definitions
Model Serving Endpoints	resources.model_serving_endpoints	ML endpoint deployment
Experiments	resources.experiments	MLflow experiment definitions
Notebooks	(referenced by jobs)	Path to notebook in workspace/repo

3.2 Bundle Project Structure

■ DAB project file structure

```
my_project/
■■■ databricks.yml          # Root bundle config (required)
■■■ resources/
■   ■■■ jobs.yml            # Job definitions
■   ■■■ pipelines.yml       # DLT pipeline definitions
■■■ src/
■   ■■■ notebooks/
■   ■   ■■■ 01_ingest.py
■   ■   ■■■ 02_transform.py
■   ■■■ sql/
■       ■■■ gold_query.sql
■■■ tests/
■   ■■■ test_transform.py
■■■ .github/workflows/
    ■■■ deploy.yml          # CI/CD pipeline (GitHub Actions)
```

4. databricks.yml — Bundle Configuration

databricks.yml is the root configuration file for a DAB. It defines the bundle name, workspace settings, targets (environments), and references to resource files.

4.1 Minimal databricks.yml

■ **Minimal databricks.yml with job**

```
# databricks.yml — root bundle config
bundle:
  name: my_etl_project

workspace:
  host: https://adb-1234567890.azuredatabricks.net

resources:
  jobs:
    bronze_to_silver_job:
      name: "Bronze to Silver ETL"
      tasks:
        - task_key: ingest
          notebook_task:
            notebook_path: ./src/notebooks/01_ingest.py
          job_cluster_key: main_cluster
        - task_key: transform
          depends_on:
            - task_key: ingest
          notebook_task:
            notebook_path: ./src/notebooks/02_transform.py
          job_cluster_key: main_cluster
      job_clusters:
        - job_cluster_key: main_cluster
          new_cluster:
            spark_version: 15.4.x-scala2.12
            node_type_id: Standard_DS3_v2
            num_workers: 2
```

4.2 Including DLT Pipeline

■ **DLT pipeline in databricks.yml**

```
resources:
  pipelines:
    medallion_pipeline:
      name: "Medallion Pipeline"
      target: catalog.main
      libraries:
        - notebook:
            path: ./src/notebooks/pipeline.py
      continuous: false
      channel: CURRENT
```

4.3 Variables — Parameterize the Bundle

■ **Bundle variables**

```
# Define variables with defaults
variables:
  catalog_name:
    default: dev_catalog
    description: "Target Unity Catalog name"
```

```
  cluster_size:
    default: "2"

# Reference variables with ${var.name}
resources:
  jobs:
    my_job:
      tasks:
        - task_key: transform
          notebook_task:
            notebook_path: ./src/transform.py
            base_parameters:
              catalog: ${var.catalog_name}
```

5. Environment Targets & Promotion

DABs use targets to define environment-specific overrides. A single bundle can deploy to dev, staging, and prod with different workspace URLs, cluster sizes, catalog names, and schedules — all defined in databricks.yml.

5.1 Multi-Environment targets Block

■ **Multi-environment targets**

```
# databricks.yml — multi-target config
bundle:
  name: my_etl_project

targets:
  dev:
    mode: development       # adds [dev username] prefix to resources
    workspace:
      host: https://dev-workspace.azuredatabricks.net
    variables:
      catalog_name: dev_catalog

  staging:
    workspace:
      host: https://staging-workspace.azuredatabricks.net
    variables:
      catalog_name: staging_catalog

  prod:
    mode: production
    workspace:
      host: https://prod-workspace.azuredatabricks.net
    variables:
      catalog_name: prod_catalog
    run_as:
      service_principal_name: prod-sp@company.com
```

5.2 Development Mode Behavior

mode: development	Effect
Resource naming	All resources prefixed with [dev] to avoid conflicts
Schedules	Job schedules are PAUSED automatically
Permissions	Creator gets owner rights
Purpose	Safe for local dev — no accidental prod schedule runs

5.3 Production Mode Behavior

mode: production	Effect
Run-as identity	Must specify run_as (service principal recommended)
Schedules	Schedules are ACTIVE
Resource naming	Exact name from config (no prefix)
Purpose	Production deployment with service principal identity

■ **Exam Focus** Exam: mode: development auto-pauses job schedules and prefixes resources. mode: production activates schedules and requires run_as identity.

6. Databricks CLI

The Databricks CLI is a command-line tool for interacting with all Databricks APIs. For CI/CD, the most important commands are in the bundle subcommand group.

6.1 Core Bundle Commands

Command	Description
databricks bundle init	Create a new bundle project from template
databricks bundle validate	Validate databricks.yml syntax and schema
databricks bundle deploy	Deploy resources to target workspace
databricks bundle deploy --target prod	Deploy to specific target (environment)
databricks bundle run	Trigger a job or pipeline defined in bundle
databricks bundle run --target staging	Run in specific target
databricks bundle destroy	Remove all deployed resources from workspace
databricks bundle summary	Show deployed resource IDs and URLs

6.2 Authentication Setup

■ **CLI authentication**

```
# Configure CLI with personal access token
databricks configure --token
# Prompts: Databricks Host + Token

# Or using OAuth (recommended)
databricks auth login --host https://workspace.azuredatabricks.net

# Verify authentication
databricks auth env

# Use service principal in CI/CD (environment variables)
export DATABRICKS_HOST=https://prod-workspace.azuredatabricks.net
export DATABRICKS_TOKEN=${{ secrets.DATABRICKS_TOKEN }}
databricks bundle deploy --target prod
```

6.3 Other Useful CLI Commands

■ **Useful CLI commands**

```
# Workspace file operations
databricks workspace ls /Repos/team/project
databricks workspace import notebook.py /Repos/team/notebook.py

# Jobs management
databricks jobs list
databricks jobs run-now --job-id 12345
databricks runs get --run-id 67890

# Clusters
databricks clusters list
databricks clusters start --cluster-id abc123

# Secrets
databricks secrets list-scopes
databricks secrets put-secret --scope my-scope --key my-key
```

7. CI/CD Pipeline Patterns

7.1 GitHub Actions CI/CD Example

■ GitHub Actions CI/CD workflow

```
# .github/workflows/deploy.yml
name: Deploy to Databricks

on:
  push:
    branches: [main]        # Deploy to prod on merge to main
  pull_request:
    branches: [main]        # Test on staging for PRs

jobs:
  deploy:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v3

      - name: Install Databricks CLI
        run: pip install databricks-cli

      - name: Validate Bundle
        run: databricks bundle validate
        env:
          DATABRICKS_HOST: ${{ secrets.STAGING_HOST }}
          DATABRICKS_TOKEN: ${{ secrets.STAGING_TOKEN }}

      - name: Deploy to Staging
        if: github.event_name == "pull_request"
        run: databricks bundle deploy --target staging

      - name: Deploy to Production
        if: github.ref == "refs/heads/main"
        run: databricks bundle deploy --target prod
        env:
          DATABRICKS_HOST: ${{ secrets.PROD_HOST }}
          DATABRICKS_TOKEN: ${{ secrets.PROD_TOKEN }}
```

QUIZ **Practice Questions** Week 10 — CI/CD, Git Folders & DABs

8. Practice Questions

1. A data engineer wants to work on a new feature without affecting the main branch. What should they do in Databricks Git Folders?

A) Edit notebooks directly on the main branch

B) Create a feature branch from main, work there, then submit a PR

C) Create a copy of the repo with a new name

D) Use a separate workspace for each feature

2. Which file is the required root configuration file for a Databricks Asset Bundle?

A) bundle.json

B) databricks.yml

C) dab_config.yaml

D) workspace_config.yml

3. A bundle target is configured with mode: development. What automatic behavior does this enable?

A) Resources are deployed to production immediately

B) Resources are prefixed with the developer username and job schedules are paused

C) The bundle is validated only — no actual deployment

D) All resources are deployed as read-only

4. A team needs to deploy the same bundle to three environments with different catalog names. What is the correct approach?

A) Maintain three separate databricks.yml files

B) Use targets in databricks.yml with variable overrides per environment

C) Use three separate Git repos for dev, staging, prod

D) Hardcode environment names in each notebook

5. Which CLI command validates the databricks.yml syntax without deploying anything?

A) databricks bundle check

B) databricks bundle dry-run

C) databricks bundle validate

D) databricks bundle lint

6. A service principal should run all production jobs. Where in databricks.yml is this configured?

A) Under resources.jobs.run_as

B) Under targets.prod.run_as.service_principal_name

C) As a job parameter: "--run-as-sp"

D) In the cluster configuration under spark_conf

7. An engineer runs "databricks bundle deploy --target prod". What does this do?

A) Validates the bundle only

B) Deploys bundle resources to the prod target workspace as defined in databricks.yml

C) Runs all jobs in the bundle on prod

D) Destroys and recreates all prod resources

8. In a CI/CD pipeline, what is the correct order of operations?

A) deploy → validate → run → test

B) validate → deploy → run → test

C) validate → test → deploy → run

D) run → validate → deploy → test

9. A bundle variable is defined as ${var.catalog_name} with default "dev_catalog". The prod target sets catalog_name: "prod_catalog". What value does a notebook receive when deployed to prod?

A) "dev_catalog" — defaults always take precedence

B) "prod_catalog" — target-level variable overrides the default

C) Both values — the notebook receives an array

D) An error — variable conflicts are not allowed

10. Which Git Folder operation fetches the latest commits from the remote repo and updates the local workspace copy?

A) Commit

B) Push

C) Pull (or Sync)

D) Clone

ANSWERS **Answer Key** Week 10 Practice Questions

9. Answer Key

#	Ans	Explanation
1	(B)	Best practice: create a feature branch from main, develop there, then submit a Pull Request to merge back to main. This protects main from in-progress changes.
2	(B)	databricks.yml is the required root configuration file for every DAB. It defines the bundle name, workspace settings, targets, and resource references.
3	(B)	mode: development auto-prefixes all resource names with [dev] to avoid conflicts, and PAUSES all job schedules to prevent accidental runs in the dev environment.
4	(B)	targets in databricks.yml define environment-specific overrides (workspace URL, variables, cluster size). A single bundle can deploy to dev/staging/prod with different catalog names via variable overrides.
5	(C)	databricks bundle validate checks the bundle configuration for syntax errors and schema violations without deploying any resources. Always run validate in CI before deploy.
6	(B)	run_as under a target specifies the identity for job execution in that environment. For production, set targets.prod.run_as.service_principal_name to the SP's application ID or name.
7	(B)	databricks bundle deploy --target prod creates or updates all resources defined in the bundle in the prod target workspace. It does NOT trigger job runs — use bundle run for that.
8	(B)	Standard order: validate (syntax check) → deploy (create/update resources) → run (trigger the job/pipeline) → test (verify results). Always validate before deploying.
9	(B)	Target-level variable overrides take precedence over bundle-level defaults. When deploying to prod, catalog_name resolves to "prod_catalog".
10	(C)	Pull fetches changes from the remote repository and merges them into the current branch in the workspace. Sync does pull + push together. Clone is for the initial setup only.

10. Week 10 Cheat Sheet

Concept	Key Facts
Git Folders	Browser + commit + push directly in Databricks workspace
Supported providers	GitHub, GitLab, Azure DevOps, Bitbucket, AWS CodeCommit
DABs (= Asset Bundles)	Define Databricks resources as YAML code
databricks.yml	Root config: bundle name, workspace, targets, resources, variables
${var.name}	Reference a bundle variable; override per target
targets:	Define dev/staging/prod environments with overrides
mode: development	Prefix resources [dev username]; PAUSE job schedules
mode: production	Exact names; ACTIVE schedules; run_as required
run_as:	Identity for job execution — use service principal in prod
databricks bundle validate	Syntax check — no deployment
databricks bundle deploy	Create/update resources in target workspace
databricks bundle deploy --target X	Deploy to specific target
databricks bundle run	Trigger job or pipeline defined in bundle
databricks bundle destroy	Remove all bundle resources from workspace
CI/CD order	validate → deploy → run → test
Branch strategy	feature → staging (PR) → main/prod (PR + auto-deploy)

BONUS **Advanced DABs Patterns** Lab & Exam Scenarios

A. Full databricks.yml with Overrides

■ **Complete multi-env databricks.yml**

```
# Complete multi-environment databricks.yml
bundle:
  name: orders_pipeline

variables:
  catalog:
    default: dev_catalog
  schedule_pause:
    default: PAUSED

resources:
  jobs:
    orders_etl:
      name: "Orders ETL — ${var.catalog}"
      schedule:
        quartz_cron_expression: "0 0 6 * * ?"
        timezone_id: "UTC"
        pause_status: ${var.schedule_pause}
      tasks:
        - task_key: ingest
          notebook_task:
            notebook_path: ./src/01_ingest.py
            base_parameters:
              catalog: ${var.catalog}
          job_cluster_key: etl_cluster
        - task_key: transform
          depends_on: [{task_key: ingest}]
          notebook_task:
            notebook_path: ./src/02_transform.py
            base_parameters:
              catalog: ${var.catalog}
          job_cluster_key: etl_cluster
      job_clusters:
        - job_cluster_key: etl_cluster
          new_cluster:
            spark_version: 15.4.x-scala2.12
            node_type_id: Standard_DS3_v2
            num_workers: ${var.workers}

targets:
  dev:
    mode: development
    workspace:
      host: https://dev.azuredatabricks.net
    variables:
      catalog: dev_catalog
      schedule_pause: PAUSED
      workers: "2"

  prod:
    mode: production
    workspace:
```

```
  host: https://prod.azuredatabricks.net
variables:
  catalog: prod_catalog
  schedule_pause: UNPAUSED
  workers: "8"
run_as:
  service_principal_name: etl-sp@company.onmicrosoft.com
```

B. DABs vs Manual Deployment Comparison

Aspect	Manual (UI)	DABs (YAML + CLI)
Repeatability	Error-prone — manual steps	Deterministic — same YAML = same result
Environment promotion	Manual re-config per env	Target overrides handle env differences
Version control	No history of job config changes	Full Git history of resource changes
Code review	Not possible for job config	PR review for any change to databricks.yml
Rollback	Manual UI changes needed	git revert + bundle deploy
Secret management	Hardcoded in UI sometimes	Reference secret scopes in YAML

C. DABs Exam Keyword Quick Map

Keyword in Question	Answer Points To
"infrastructure as code" for Databricks	DABs (Databricks Asset Bundles)
"define job in YAML"	databricks.yml resources.jobs section
"deploy to different environments"	targets: block with variable overrides
"schedules auto-paused in dev"	mode: development behavior
"service principal for prod jobs"	targets.prod.run_as.service_principal_name
"validate bundle syntax"	databricks bundle validate
"deploy resources to prod"	databricks bundle deploy --target prod
"trigger job defined in bundle"	databricks bundle run
"remove all deployed resources"	databricks bundle destroy
"create new bundle from template"	databricks bundle init
"CI/CD order"	validate → deploy → run → test
"branch for development"	Git Folders feature branch strategy
"PR triggers staging test"	GitHub Actions on: pull_request workflow

D. Phase 4 (Wk9+10) Self-Assessment

Week 9 — Lakeflow Jobs

- ■ I know 8 task types; SQL task needs SQL Warehouse
- ■ I can model fan-out/fan-in DAGs with depends_on
- ■ I know run_if values: ALL_SUCCESS/ALL_DONE/NONE_FAILED/AT_LEAST_ONE_FAILED
- ■ I know outcome values on depends_on: SUCCEEDED/FAILED/TIMEDOUT/SKIPPED
- ■ I can configure If/Else and For-Each tasks
- ■ I know 3 trigger types: scheduled/file arrival/table update

Week 10 — CI/CD & DABs

- ■ I know databricks.yml structure: bundle/variables/resources/targets
- ■ I understand mode: development (pause, prefix) vs mode: production (active, run_as)
- ■ I can write multi-environment targets with variable overrides
- ■ I know CLI commands: validate/deploy/run/destroy
- ■ I can describe a complete CI/CD pipeline: PR → staging → prod
- ■ I know Git Folders operations: clone/branch/commit/push/pull/PR

■ **Exam Focus** Phase 4 = Domain 4 (16%) + Domain 5 (10%) = 26% combined. Focus on DAG control flow (Wk9) and DABs deployment model (Wk10) — high-yield exam topics.

E. Phase 4 Final Key Facts

Topic	One-Line Fact
SQL task compute	SQL task = SQL Warehouse ONLY; Notebook task = cluster
Job cluster vs all-purpose	Job cluster = ephemeral + cheap (prod); All-purpose = persistent + expensive (dev)
depends_on default	Downstream task runs only after upstream SUCCEEDED
outcome="FAILED"	Downstream task runs ONLY when upstream fails — error handler pattern
run_if="ALL_DONE"	Always runs regardless of upstream outcome — cleanup/notification
run_if="NONE_FAILED"	Runs if no upstream failed (includes skipped tasks)
If/Else task	op + left + right; branches use outcome="true" or outcome="false"
For-Each concurrency	concurrency=1: sequential; concurrency=N: N parallel iterations
File arrival trigger	min_time_between_triggers_seconds = debounce window
Table update trigger	Monitors Delta transaction log; ANY_UPDATED or ALL_UPDATED
{{input}} in For-Each	Current iteration value in child task parameters
databricks.yml	Root config for DABs — required file
mode: development	Auto-prefix resources + PAUSE schedules
mode: production	Exact names + ACTIVE schedules + run_as required
bundle validate	Syntax check only — no resources deployed
bundle deploy	Create/update resources in target workspace
bundle run	Trigger a specific job or pipeline by resource key
bundle destroy	Delete all bundle resources from target workspace

F. CI/CD Complete Exam Reference

Scenario	Correct Answer
"Define Databricks job as code"	DABs — databricks.yml resources.jobs
"Deploy same job to dev and prod"	targets block with variable overrides per environment
"Prevent job running in dev env"	mode: development auto-pauses schedules
"Use service principal in prod"	targets.prod.run_as.service_principal_name
"Check YAML syntax before deploy"	databricks bundle validate
"Deploy resources to staging"	databricks bundle deploy --target staging
"Trigger job from CLI"	databricks bundle run --target prod
"Remove all deployed resources"	databricks bundle destroy --target prod
"Version control notebooks"	Git Folders — commit and push to remote repo
"Review notebook changes before merge"	Pull Request in GitHub/GitLab/Azure DevOps
"CI/CD pipeline order"	validate → deploy → run → test
"${var.catalog_name} resolves to?"	Value from target variables override, else bundle default
"DABs = ?"	Declarative Automation Bundles (also called Databricks Asset Bundles)

■ **Exam Focus** Domain 5 (CI/CD) = 10% of exam. DABs structure, targets/mode behavior, and CLI commands (validate/deploy/run/destroy) are the most-tested concepts.